



- 請先開啟 VS Code並安裝好環境
- 開啟ChatGPT，建立一個專案Project，名稱為 vibe-coding-week2，於新增檔案裡上傳今日的講義

你其實已經在 「控制 AI」 只是你還沒發現

如果你沒有先把這件事對齊，那 Week 2 一開始打造產品時，你會很容易出現

我好像都懂，但 AI 一改就歪？怎麼功能一多，整個系統就亂了？是不是我指令下得不夠好？

Week 1

我們到底做了什麼？

用產品語言重新看一次

表面上你做的是

- 寫 Spec 文件
- 定義 Schema 結構
- 畫出 User Flow
- 用 AI 做出可運作的小系統

但本質上你完成的是

- 建立系統邊界
- 定義行為路徑
- 掌握控制權
- 管理 AI 的工作方式



Week 1 的三個關鍵成就



第一件事：畫出邊界

透過 Schema，你告訴 AI 哪些資料存在、哪些現在不准出現。

這一步的本質是：限制 AI 的想像空間。



第二件事：定義路徑

透過 User Flow，你建立了一張「出事時可以回頭看的地圖」。

當系統怪怪的，你知道該回到哪一步檢查。



第三件事：握住控制權

Spec 文件的真正功能，不是記錄需求，而是讓 AI 在一個被限制、被定義過的世界裡工作。

Spec-driven 到底在驅動什麼？

Week 1，我們一直說這門課是 Spec-driven

Week 2，你會開始看到另一個詞：Context-driven。很多人會在這裡開始混亂，所以我們直接一次講清楚。

- ❏ AI 永遠是 Context-driven。
Spec-driven 不是取代它，而是用來「設計 Context」。



什麼是 Context ？

- 1 專案結構：目前的資料夾結構與檔案組織
- 2 檔案內容：每一個檔案的完整內容
- 3 修改歷程：你剛剛做了哪些修改
- 4 當前指令：你現在下了什麼指令
- 5 活動區域：哪些地方是最近被動過的

對 AI IDE（包含 Amazon Q）來說，AI 不是只看你這一句話，而是看「整個專案目前長什麼樣子」。

Spec 的核心任務

讓 AI 每次看到的 Context 都長得差不多

你在 Week 1 其實已經做了三件事：**壓縮 Context，穩定 Context，鎖定 Context**，只是你當時還沒把這件事「講清楚」。

Week 2 容易「爆掉」？

Week 1 的資料是靜態的

- 新增資料
- 顯示資料
- 刪掉再來

Context 變化很小，AI 比較不容易失控。

Week 2 的資料會移動

- 同一筆資料
- 會因為狀態不同
- 出現在不同位置
- 被不同邏輯處理

Context 會頻繁變動，而且是連續變動。

1 靜態資料

2 動態狀態

3 持續變動

請暫時放下「我是新手」這個身份

從 Week 2 開始，你的角色會變成：正在打造一個產品雛形的人。

- 1 你做的每個決定，都會影響後面能不能上線
- 2 你加的每個功能，都可能影響系統穩定性
- 3 你必須開始「選擇不做什麼」

別把這當作壓力，這是你開始真的在做產品的證據。



Week 2 的工作方式

- 1 Context 隔離：Week 1 的專案，不要混進來
- 2 統一世界：Week 2 的所有嘗試，都在同一個世界裡完成
- 3 問題追蹤：出問題時，你知道你是在哪一週的 Context

這不是格式規定，而是 Context 管理的一部分。

📌 在本週所有實作中，我們會統一使用：`vibe-coding-week2`

從這一章開始使用 Amazon Q

從 Week 2 開始，你會明顯感覺到：不能再用「幫我做一個 XXX」這種指令了。

Chapter 0～Chapter 2 的使用原則

✓ 理解現況

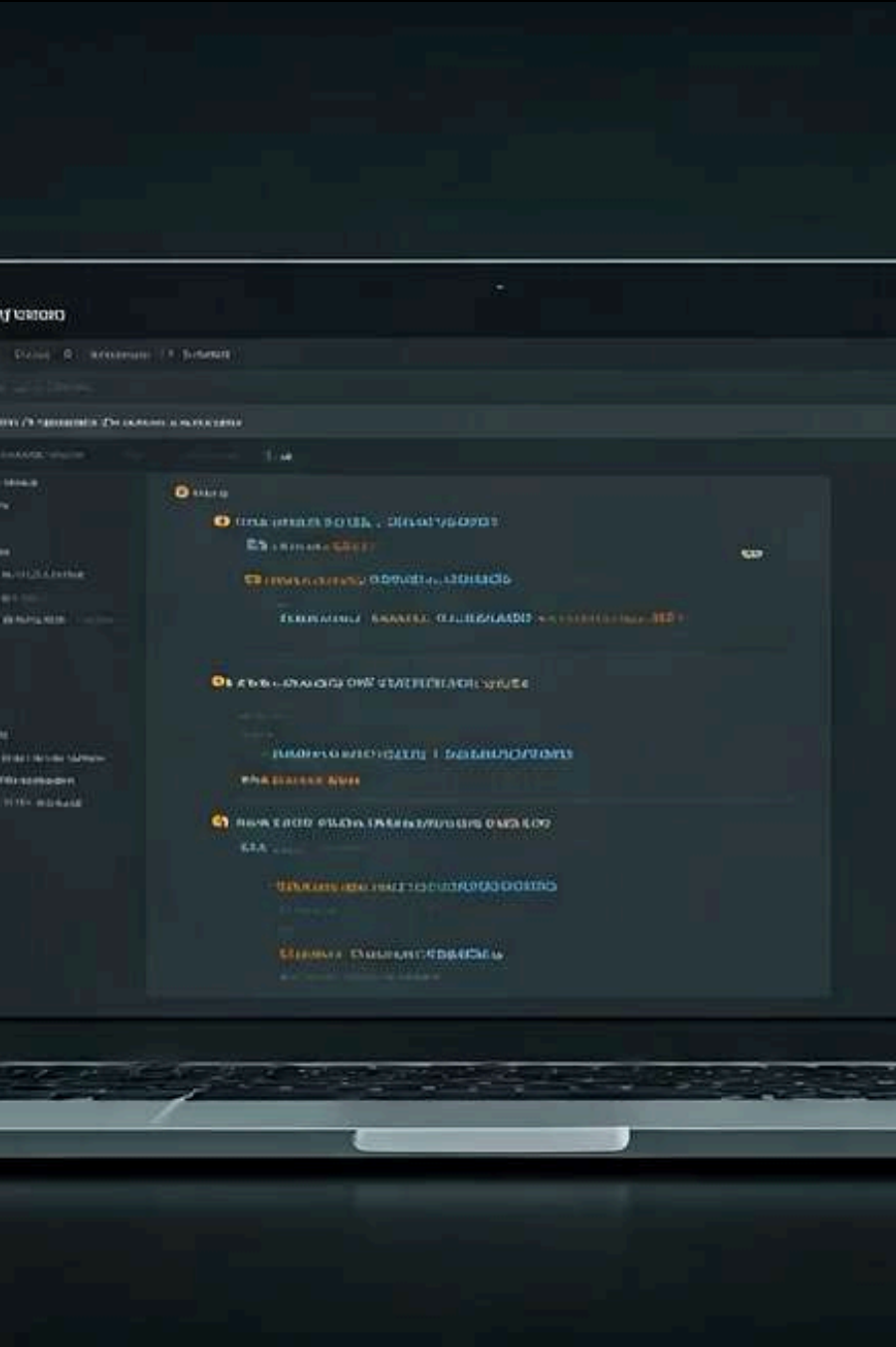
幫我總結目前專案的結構與資料流

✓ 對齊理解

依照 Spec 確認你的理解是否正確

✗ 暫不實作

這階段不讓 AI 動手，只確認 Context





你現在應該能回答這三個問題

- 1 你不是來「多寫功能」的
- 2 你是來「讓系統開始穩定運作」的
- 3 Spec 不是作業，是你的煞車系統

接下來，我們會從一個產品一定會遇到的問題開始：為什麼只要系統想被使用，就一定會出現「狀態」？

Chapter 1

產品為什麼一定會有狀態

從「能跑的系統」，走向「會被使用的產品」

在 Week 1，你已經成功做出一個可以運作的系統，它可以新增資料、顯示資料，而且流程是通的。

但從 Week 2 開始，我們要面對一個新的現實：只要一個系統想被「持續使用」，它就一定會出現「狀態」。



為什麼沒有狀態的系統，只能是 Demo

先回想一下你在 Week 1 做的事情：輸入資料、存起來，最後顯示出來。

這樣的系統，有沒有用？有。但它有一個很明顯的限制：它不知道事情「現在進行到哪裡」。

Demo 系統：關心的是「資料有沒有成功存進去」

產品系統：關心的是「這件事現在卡在哪一個階段」

狀態不是資料的屬性 而是「行為的階段」 一件事情「現在的立場」

狀態的生活化例子

1 一篇內容

- 還沒開始寫
- 正在編輯中
- 已經完成

2 一個申請

- 已送出
- 審核中
- 已通過

3 一個任務

- 待辦
- 進行中
- 已完成

為什麼「狀態一出現」，Context 就會變危險

在沒有狀態的系統裡：資料新增一次、顯示一次、Context 變化很小。

但只要你引入狀態，事情就不一樣了，狀態讓系統發生三件事：



資料會開始「移動」

同一筆資料，可能會原本顯示在 A 區，狀態改變後出現在 B 區，在 A 區消失。



畫面開始依賴邏輯

你看到的不是「全部資料」，而是「符合某個狀態的資料」。



Context 會頻繁變動

每一次狀態切換，都會改資料、改畫面、改流程。如果這些規則沒有先寫清楚，AI 就會開始補邏輯、補流程、補假設。

這就是為什麼App #2從狀態開始

在 Week 2，我們選擇的練習專案是：

App #2 內容管道儀表板 Content Pipeline Board

它表面上看起來像一個看板，但真正的學習重點只有一個：

如何在「狀態不斷變動」的情況下，讓系統還是穩定的。



這個 App 會逼你面對三件事

1 唯一真實狀態

一筆資料只能有一個真實狀態

2 同步更新

狀態改變，畫面就必須跟著改

3 允許的轉移

所有狀態轉移，都必須是「被允許的」

這正是產品世界每天都在發生的事，而在本章你「不需要」先會的事情：

我們刻意不談

- 複雜的前端狀態管理
- 框架或套件
- 效能或最佳化
- 拖拉或動畫

我們只關心

- 狀態是否清楚
- 資料是否一致
- Context 是否可控



你現在應該能回答這三個問題



產品必有狀態



狀態讓系統易亂



必須用 Spec 管控

如果你現在能回答這三個問題，那代表你已經準備好進入下一章。

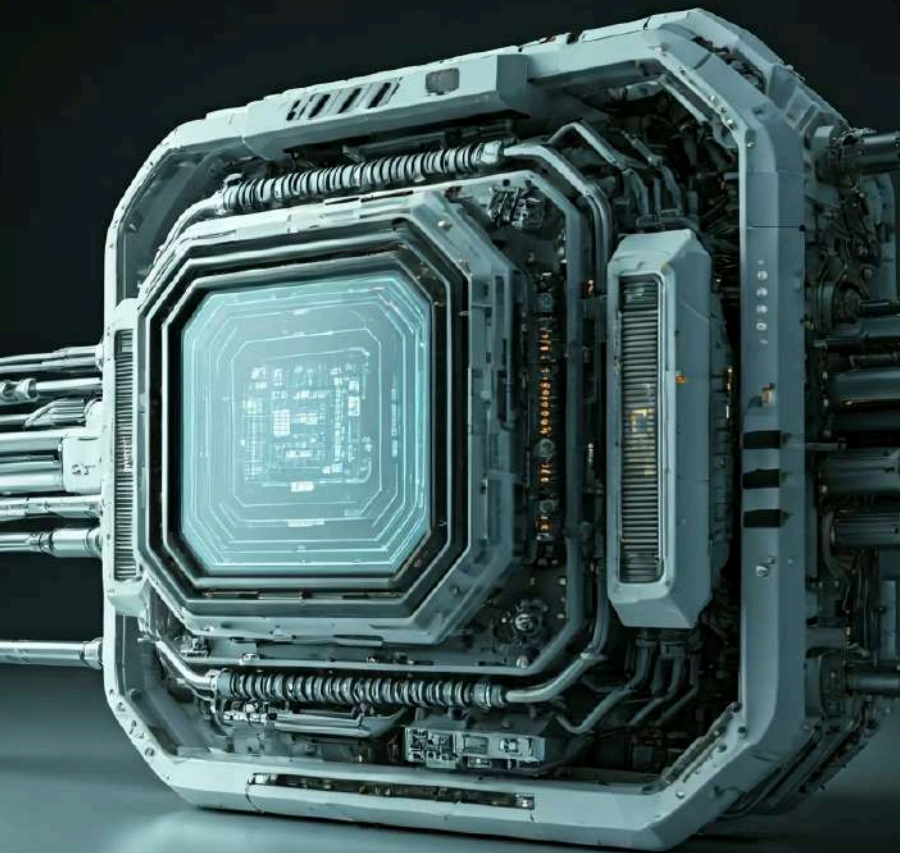
Chapter 2

Context-driven 的真相

在 Chapter 2，我們要正面拆解一個問題：

當系統開始像產品一樣成長時，AI 為什麼會特別容易失控？

為什麼系統一開始像天使，後來卻越改越失控？



先破除一個迷思：AI 沒有「記憶斷點」

很多人以為 AI 這樣工作

1. 我現在下指令
2. AI 只針對這個指令處理
3. 做完就結束

但事實完全不是

AI IDE 的真實工作方式：

我現在看到的所有東西，都是我做判斷的依據。

AI 看到的「所有東西」包含什麼



專案結構



檔案內容



修改歷程



當前指令



操作脈絡



沒有重來鍵，也沒有自動清空 Context



為什麼要先準備 Demo 專案？



有資料夾結構



有初步 Spec



有狀態欄位



但沒有完整邏輯

這不是偷懶，是刻意設計

Context 不是等你寫完系統才出現

而是從你建立第一個檔案就開始存在。

📁 Demo 專案的用途：讓你跟 AI 站在同一個「世界畫面」裡

請下載 [demo.zip](#) 至 vibe-coding-week2 資料夾，解壓縮。

產品型專案為什麼 Context 容易失控？

因為產品型專案，跟小練習有三個根本差異。



差異一：檔案數量快速增加

一開始只有一個 HTML、一個 JS、一個資料結構。

但一進入產品思維，很快就會出現多個邏輯檔、多個狀態處理、多個顯示區塊。



差異二：頻繁回頭改

產品不是一次寫完的。

你一定會改流程、改狀態、改命名、改結構。這代表 AI 看到的是「被動過很多次的世界」。



差異三：狀態會互相影響

在產品裡，改一個狀態往往會影響三個畫面、牽動兩條流程。

Token 爆炸的真相

1 「幫我調整一下狀態顯示」

對你來說是小改，對 AI 來說卻是：我是不是要重新推理所有可能影響到的地方？

2 「我想稍微改一下流程」

於是它開始回顧整個專案、嘗試補齊缺失邏輯、預防你沒講清楚的情境。

3 「這裡怪怪的，幫我修一下」

結果就是：Token 飆高、修改範圍失控、改 A 壞 B。

💡 Token 爆，不是因為你話多，而是因為 AI 必須重新理解整個世界



Context-driven不是問題 失控才是

問題從來不是：AI 太聰明、AI 愛亂改、AI 不聽話

真正的問題是 **Context 沒有被人類設計過**

❑ Context-driven 不是缺點，而是 AI 能工作的前提

沒有 Spec 的 Context 會發生什麼事？

1 AI 只能猜意圖

2 世界觀重建

3 推理成本增加

4 穩定性下降

Spec-driven 的真正角色

不是命令，是護欄

Spec 不是拿來叫 AI 做事的，Spec 是用來限制 AI「怎麼想事情」的

Spec 在 Context 中扮演的三個角色

- 1 縮小思考空間
- 2 固定世界觀
- 3 標示禁止區域



Chapter 2 關鍵洞察

- 1 產品一定會有狀態：這是產品的本質特性
- 2 狀態會讓Context快速變動：需要主動管理
- 3 Context-driven本身沒錯：沒被管理才會失控



你現在應該能回答這三個問題

在進入下一章之前，你應該開始能判斷這些狀況：

“

現在 AI 是在「理解清楚後修改」
還是在「邊猜邊補」？
這次修改，是不是動到太多 Context？
我是不是該先停下來補 Spec？

”

如果你開始有這些念頭，代表你已經從「使用者」進化成「管理者」。

帶著這個理解，進入 Chapter 3

那 Spec，到底要怎麼寫，才能真的「管得住 Context」？

到目前為止，我們已經確認三件事：

- 產品一定會有狀態
- 狀態會讓 Context 快速變動
- Context-driven 本身沒錯，沒被管理才會失控

從理解到實踐的關鍵轉折

到目前為止你學到的

- 為什麼產品需要狀態
- 為什麼 Context 會失控
- 為什麼需要 Spec 管理

你現在的位置



接下來你要學的

- 如何設計有效的 Spec
- 如何用 Spec 控制 Context
- 如何讓系統穩定成長

☐ 你已經完成了最重要的認知轉換，接下來就是將理解轉化為實踐能力



準備好了嗎？

讓我們開始真正掌控 AI

從這一刻起，你不再是被動地使用 AI 工具，而是主動地設計 AI 的工作環境。

記住：Spec 不是限制，而是讓你的創意能夠穩定實現的基礎建設。

Chapter 3

Spec-driven 的真正用途

Spec 不是文件，而是你用來「接管 Context」的控制器。

在接下來的內容中，我們將學習如何讓 Spec 從說明文件，正式升級成 Context 的控制器。

你在 Chapter 2 完成的重要里程碑

第一次看清真相

你第一次看清楚了 AI 現在是怎麼理解你的專案。這是一個關鍵的覺醒時刻，讓你開始掌握與 AI 協作的主導權。

發現關鍵事實

AI 並沒有亂來，它只是「補齊了你沒說清楚的地方」。理解這一點，是從被動接受轉向主動控制的起點。



問題變得具體了

1 識別問題

AI 會自動補齊未明確定義的部分

2 形成疑問

那我要怎麼寫 Spec，才能讓 AI 不用猜？

3 找到方向

透過精確的 Spec 來控制 Context



校正一個錯誤期待：Spec 不是一次寫完的

- ❏ 很多人聽到 Spec-driven，腦中會出現「先寫一份超完整 Spec，再交給 AI 一次做完，中途不再動」的畫面。這個想像，在產品世界裡幾乎不可能成立。

產品型 Spec 的真實樣子

1 一開始很小

從核心概念開始，建立最小可行的規格說明

2 隨系統成長而補齊

根據實際需求和問題逐步完善

3 每次只解決現在的 Context

專注於當前階段，不過度設計未來

在 Week 2，我們用的心法是：Spec 不是藍圖，而是「護欄」。

Spec 要控制的核心目標不是控制功能，而是「理解範圍」

在 Chapter 2，你已經讓 Amazon Q 幫你盤點過哪些地方是明確的、哪些地方是它必須自行推測的。

Spec 的第一個任務，就是把「必須推測的地方」一個一個消掉。

用 App #2 的例子來看

1 目前已有的部分

- 狀態欄位 (status)
- 三個狀態名稱
- 初步資料結構

2 還沒寫清楚的部分

- 狀態可以怎麼轉？
- 哪些轉移是不允許的？
- 狀態改變時，畫面應該發生什麼事？

這些「沒寫的地方」，就是 Context 最容易失控的地方。

區塊一：系統目標 (System Goal)

1 用一句話回答

這個系統「現在」要解決什麼問題？

2 注意「現在」兩個字

Week 2 的目標不是未來藍圖

- 📄 • 管理內容狀態，確保資料一致，支援後續擴充

區塊二：資料與狀態定義 (Data & State)

不是資料庫設計，而是語意設計

你要清楚定義資料的意義和狀態的規則，而不只是技術結構。

你要回答的問題

- 一筆資料，最少包含什麼？
- status 有哪些合法值？
- 是否允許自由新增狀態？

💡 如果這裡沒寫清楚，AI 就一定會「好心幫你補」。

區塊三：允許的流程（Allowed Flow）

這一段非常關鍵，因為它直接影響 AI 的修改行為。

- 1 明確寫出哪些狀態可以轉到哪些狀態
- 2 定義哪些轉移是禁止的
- 3 確保一次操作只做一件事

這會大幅降低「牽一髮動全身」的修改。

區塊四：不在本週範圍內（Out of Scope）不是偷懶，而是保命

明確寫出這一週不做什麼、哪些功能暫時不碰、哪些結構不能動。

📌 重要提醒：Spec 不寫「不做什麼」，Context 一定會膨脹。

把 Spec 寫回 demo 專案裡：Spec 必須存在於專案本身

1 不是寫在腦中

想法必須具體化

2 不是寫在投影片裡

需要可執行的文件

3 成為 Context 的一部分

AI 以後每次看專案，都會看到這份 Spec

```
vibe-coding-week2/  
└─ spec/  
   └─ app-spec.md
```

Spec 對齊實測：讓 Amazon Q 幫你照鏡子 p.38

1 先準備一份刻意不完美但可用的 Spec

建立 app-spec.md 並填入範例內容

2 執行 Amazon Q 對齊指令

讓 AI 說明它的理解

3 檢視回覆判斷對齊成功與否

確認 AI 看到的世界跟你腦中是同一個

對齊成功的訊號



使用相同語言

AI 用跟你 Spec 幾乎一樣的語言描述系統



遵守狀態規則

不會新增新的狀態名稱



清楚禁止項目

明確說出哪些轉移被禁止



尊重範圍限制

指出 Out of Scope，不會自行補功能

危險訊號：Spec 還沒鎖住 Context

⚠ 開始反問問題

「Can items move back to previous states?」

⚠ 提出額外建議

「It might be useful to allow editing...」

⚠ 自行做出假設

「Users may want to add custom statuses...」

💡 這些不是 Q 亂來，而是：你的 Spec 還留了太多「可以合理推測」的空間。

這時候該做什麼？

1 回到 [app-spec.md](#)

2 補一句「不允許」

3 再跑一次對齊指令

不是改程式，而是先修正 Spec，確保 AI 的理解範圍被正確限制。

這個實測在教你什麼？

剛剛這個過程，AI 其實完全沒有寫任何程式碼，但你已經做到三件事，這正是 Spec-driven 的真正價值。

1. 降低 Context 的不確定性
2. 防止 AI 提前補功能
3. 為後續實作建立安全邊界

為什麼「Spec 對齊」一定要在寫程式前？

1 實作開始後

檔案數會變多

2 複雜度上升

Context 會變複雜

3 成本增加

修改成本會變高

❏ 如果在這之前沒有對齊，你之後每一次 Debug，本質上都是在補 Spec。

如果 Q 開始「問很多問題」，那是好事

很多學生在這一步會很緊張：它怎麼問這麼多？是不是我 Spec 寫得很爛？

其實剛好相反。AI 問問題，代表它不敢亂猜了，這正是 Spec 開始發揮作用的訊號。

Chapter 3 關鍵不是寫 Spec，而是「測 Spec」



不只是文件技巧



而是一種能力



判斷 Spec 是否管得住
Context

當你能回答這個問題，你就已經準備好進入 Chapter 4。



你現在應該能回答這三個問題

- 1 你知道 Spec 在控制哪一段 Context
- 2 你知道哪些地方「還不能動」
- 3 你敢拒絕 AI 幫你加功能
- 4 你開始用 Spec 來踩煞車

如果你有這些感覺，代表你已經成功升級。

Chapter 4

產品級三原則

Backend First

× Iterative

× Atomic Edit

讓系統「開始成長」，但不爆炸



現在你的專案裡已經有三種東西

可被 AI 讀取的結構

初步但明確的 Spec

被刻意限制的 Context

產品不是寫壞的，是改壞的

為什麼產品一進實作就容易炸？

在 Week 2，你的系統會開始出現這些特徵：

系統特徵

- 有狀態
- 有多個畫面區塊
- 有重複修改
- 有「之後可能會再加」的誘惑

這代表什麼？

Context 會開始高速變動。

如果你沒有一套「改動紀律」，AI 會很努力幫你補齊，但補出來的通常不是你要的產品。

原則一：Backend First 依賴順序控制

這不是技術選擇，是「思考順序」

先把「資料與狀態的真相」定下來，再讓畫面去反映它。

- ❏ Backend 不是伺服器、不是雲端，而是資料結構、狀態規則、和資料語意。

為什麼一定要先定 Backend？

1 如果你先動 UI

AI 不知道資料從哪來

2 產生困惑

不知道哪些狀態合法

3 開始自行補充

就會開始自己補 API、補邏輯

4 結果

畫面看起來對，但資料早亂了

Backend First 的實作心法

每次你想改畫面前，先問自己一句：

我現在改的，是「資料的真相」，還是「顯示方式」？

如果是真相

先改資料與 Spec

如果只是顯示

才動畫面

原則二：Iterative 小步快跑，控制變動幅度

AI 不是不能做很多事，是不能「一次做很多事」，當你開始實作時，最危險的一句話是：「我們一次把這整段都做好吧。」

為什麼大改會害死產品？

- 1 多個 Context 同時變動
- 2 推理成本暴增
- 3 無法判斷哪個變動才是你要的
- 4 結果：什麼都做一點
- 5 但沒有一個地方是真的對



原則三：Atomic Edit 控制傷害範圍

產品不是被「寫壞」，而是被「順手改壞」

最常見的災難級指令是：「幫我修一下這裡。」對你來說很小，對 AI 來說卻是：「那我幫你重構整個世界。」

Atomic Edit 的三個必要元素

- 1 明確指定檔案
- 2 明確指定修改位置
- 3 明確限制影響範圍



三原則其實是在做同一件事



Backend First：控制「依賴順序」



Iterative：控制「變動節奏」



Atomic Edit：控制「傷害範圍」



核心目標：限制 Context 同時變動的範圍

如果你違反其中一個會發生什麼事？

違反 Backend First

UI 看起來正常，資料實際已經不一致

違反 Iterative

每次改完都不知道是哪裡壞，
Debug 成本暴增

違反 Atomic Edit

改一行，壞三個地方，AI 越幫越忙

從產品定位到系統文件的橋樑

1 Week 1：建立基礎架構與 Context 管理機制

2 Week 2 前半：實作核心功能並學會保護系統

3 Chapter 5：確立產品定位與使用者場景

4 Chapter 6：收斂成可交付的完整 Spec

每個階段都是下一階段的基礎，缺一不可。



你現在應該能回答這三個問題

- 1 穩定的 Context 管理：你已經知道如何用 Spec 控制 AI 協作範圍
- 2 可控的修改節奏：懂得在改動時保護系統完整性
- 3 明確的產品定位：清楚知道為誰開發、解決什麼問題？
- 4 下一步：可交付的 Spec：將這些知識收斂成真正可教、可傳承的文件

Chapter 6

App #2 教學用 Spec

用 Spec 把會爆炸的想法變成可完成的產品

從這一章開始，我們將學會一個重要技能：不是寫更多程式碼，而是把產品定義成「一個做得完的東西」。

這份 Spec 不是工程文件，而是用來限制 Context，控制實作節奏，確保交付的工具。



Spec 的三個使用原則

1 刪除灰色提示

所有範本中的灰色提示文字都必須刪除，換成你自己的答案。這不是填空題，而是產品定義。

2 必填欄位優先

所有標註為「A 標準必填」的欄位，下課前一定要完成。這些是產品能否站穩的關鍵。

3 避免模糊空間

沒寫清楚的地方，AI 一定會幫你「合理補完」。但那不是你的產品決策，是 AI 的猜測。

專案檔案結構

請在專案中建立以下檔案結構，這份檔案將成為 App #2 的單一真相來源（Single Source of Truth）：

```
vibe-coding-week2/  
└─ spec/  
    └─ app-spec.md
```

📁 這個檔案位置是全班統一的標準，請勿更改路徑。統一的結構讓協作與交接更容易。

第一：System Goal 系統目標

為什麼這麼重要？

系統目標不是寫給技術人員看的，而是用來回答：「這個產品現在要解決什麼問題？」

請用 2-3 行說明清楚：這個 App「現在這一版」要幫誰解決什麼問題。

填寫範例

1. System Goal

This app is designed for
[目標使用者].

The main problem it solves is
[核心問題].

For Week 2, this app focuses on:
[本週重點]

第二：User & Core Scenario 使用者與核心場景

想像一個人正在打開這個 App。他為什麼現在需要它？在什麼情境下會使用？這個場景必須具體、可想像。

不要寫「使用者想要管理任務」，而是寫「當使用者在會議中收到新工作項目時，他想要立即記錄並分類優先順序」。

第三：Data Structure 資料結構

這是整個產品的骨架。請記住一個核心原則：不要設計資料庫，只定義「最小可用資料」。

id: number

每筆內容的唯一識別碼，用於追蹤與更新

title: string

內容的標題或描述，使用者可見的主要資訊

status: string

當前狀態值，決定內容在哪個欄位顯示

📌 Week 2 不包含其他欄位。這個限制是刻意的，不是偷懶。

第四：State Definition 狀態定義

這一段是整個 App #2 的核心。狀態定義決定了產品的行為邊界。

允許的狀態值

- todo（待處理）
- in-progress（進行中）
- done（已完成）

絕對規則

不允許任何其他狀態值存在。這不是技術限制，而是產品決策。

第五：Allowed State Flow 允許的狀態轉移

1 todo

待處理的新項目

2 in-progress

正在執行的工作

3 done

已完成的項目

請非常明確定義允許與禁止的轉移，因為 AI 會完全照這裡的規則行事。每個動作只更新一個項目。

禁止的狀態轉移

1 todo → done

不允許跳過「進行中」階段直接完成

2 done → 任何狀態

已完成的項目不允許再改變狀態

3 逆向轉移

Week 2 不支援任何狀態回退功能

第六：UI Scope 畫面範圍

UI 不是討論畫面美不美，而是定義「顯示哪些資訊」與「支援哪些操作」。

包含的功能

- 三個欄位：To Do、In Progress、Done
- 每個項目只出現在一個欄位
- 狀態改變時立即更新列表
- 新增項目的輸入介面

不包含的功能

- 拖拉操作（Drag and drop）
- 動畫效果
- 進階樣式設計
- 編輯項目內容

第七：Out of Scope 本週不做的事

這一段是保命用的。明確說出「不做什麼」，比說「做什麼」更重要。



使用者認證

不實作登入、註冊或權限管理



資料庫持久化

資料只存在記憶體中



內容編輯

不支援修改已建立的項目



進階功能

不包含搜尋、篩選或排序

第八：Success Criteria 完成標準

下課前，你要能「驗收」你的作品。這些標準必須可測試、可驗證。

1 新增功能

使用者可以新增一個項目到 To Do 欄位。

2 狀態推進

使用者可以按步驟將項目移動到 Done。

3 資料一致性

項目永遠不會同時出現在多個欄位。

4 規則驗證

禁止的狀態轉移會被系統阻止。

停下來想一件事

剛剛你沒有寫任何一行程式碼，但這個 App 已經被「鎖定成一個可完成的產品」。

這就是 Spec-driven 開發的核心價值：在動手之前，先確定產品的邊界、規則與交付標準。

為什麼使用學員填空版？



知道哪裡該填

理解每個欄位的產品意義



知道哪裡不能亂填

辨識哪些是核心約束條件



知道填錯的後果

理解一個錯誤決策會如何影響整個產品



你現在應該能回答這三個問題

1 使用者定義

這個 App 是給誰用的？

2 核心問題

它現在解決哪一個核心問題？

3 範圍限制

為什麼有些功能「現在不能做」？

4 部署準備度

如果要部署，這個 Spec 已經站得住腳嗎？

如果你能做到這些，你已經不是在「學寫 App」，而是在**打造產品底盤**。

Chapter 7A

本地展示與交付

讓 App #2 可 Demo、可驗收、可交接

為什麼 Week 2 不急著部署？

到這一章為止，你要先接受一個事實：不是所有產品，都應該在「第一次完成」就急著上線。

部署會帶來什麼？

- 帳號與權限設定
- 指令與工具學習成本
- 非產品本身的 Debug

這些會造成什麼影響？

- 打斷產品思考節奏
- 稀釋核心學習重點
- 模糊狀態與資料一致性



請記住Week 2 的完成，不是「上線」，而是「可驗收」

1 本地運作

使用 Live Server 在本地端正常運作。

3 資料一致

卡片永遠只存在於一個欄位。

2 核心功能

可以新增內容並依規則推進狀態。

4 規則驗證

禁止的狀態轉移會被阻止。

本地展示方式

在 Week 2，展示方式統一採用 VS Code + Live Server 或任何等效的本地 HTTP server。

● 專案資料夾命名

確認資料夾名稱為 vibe-coding-week2

● 操作流暢

操作流程不需要重新整理頁面

● 檔案可開啟

index.html 可正常開啟並顯示。

● 即時更新

狀態改變後，列表即時更新

Demo Script 展示腳本

請練習用 60-90 秒展示你的產品。清晰的展示不是炫技，而是證明你的 Spec 真的在控制產品行為。

1 一句話介紹（10 秒）

說明這是什麼產品、給誰用、解決什麼問題

2 新增內容（20 秒）

示範新增一筆內容，確認出現在 To Do

3 推進狀態（30 秒）

展示 To Do → In Progress → Done 的流程

4 規則說明（20 秒）

說明一條「不能做的事」及其原因

交付內容清單

Week 2 的交付不是一個網址，而是一個可交接的產品資料夾。

必要檔案結構

```
vibe-coding-week2/  
├─ index.html  
├─ src/  
├─ spec/  
|   └─ app-spec.md  
└─ README.md
```

README.md 建議內容

你的 README 至少要說清楚三件事：

- 這是什麼 App
- 目前支援哪些行為
- 哪些功能刻意不包含

這會直接影響下一週「要不要部署」的決策。

為什麼交付資料夾很重要？

因為產品在進入部署前，必須先能夠：

被別人打開

檔案結構清晰，任何人都能啟動

被別人理解

有完整的 Spec 與說明文件

被別人接手

規則明確，可以繼續開發

如果交出去的東西只有畫面沒有規則，只有功能沒有 Spec，那就算上線也很快會失控。



你現在應該能回答這三個問題

1 產品定位

這個 App 是給誰用的？

3 範圍控制

哪些功能是刻意不做的？

2 核心價值

它現在解決哪一個核心問題？

4 上線準備

如果要上線，還缺哪一塊？

如果你能清楚回答，代表你已經完成 Week 2 的任務。

Chapter 7B

上線與商業化準備

給已經準備好往前衝的你

部署不是只有一種路

有些學員在 Week 2 結束時會出現這個狀態：我做完了，而且我知道下一步想幹嘛。這一節就是為你準備的。

路線一：Firebase Hosting

Week 3 全班主線

- 適合快速上線
- 幾乎零後端知識
- 有免費層級
- 非常適合 App #2 的 MVP

路線二：Cloud Run

進階選配示範

- 適合需要後端邏輯
- 彈性高，可擴展
- 需要 GCP 專案設定
- 有帳務管理要求

為什麼先選 Firebase Hosting ?



快速上線

幾分鐘內就能部署。



立即分享

獲得一個可分享的網址。



收集回饋

可以收集使用者意見。



心流不中斷

保持產品開發節奏。

產品在早期最需要的是驗證，而不是複雜架構。Firebase Hosting 完美符合靜態前端、快速部署的需求。

Cloud Run 的產品角色

Cloud Run 不是拿來「取代」Firebase 的，而是當你的產品開始需要「邏輯與彈性」時的舞台。

你現在不一定需要它，但你應該知道它存在，並理解它在產品成長路徑中的位置。

Week 3 將做的事

- 1 部署上線：把 App #2 部署到 Firebase Hosting，取得可分享的網址
- 2 理解影響：理解部署如何影響產品決策與開發節奏
- 3 商業討論：討論商業模式可能性，例如內部工具、個人 SaaS、客戶專案或訂閱制

給進階學員的建議

如果你現在已經做到 App #2 穩定、Spec 清楚、想擴充功能，那你已經站在可以開始「做產品實驗」的位置了。



實驗思維

用小範圍測試來驗證假設，而不是一次做大改動。



數據驅動

開始思考如何收集使用數據來指導產品決策。



漸進升級

一次只加一個功能，確保每次升級都在控制之下。

Chapter 8

狀態升級與互動進階

當產品開始被使用，互動為什麼一定要升級？

在 Chapter 6，你已經有一個穩定、可驗收、可交付的產品 MVP。它現在可以正確顯示狀態、保證資料一致、防止不合法的轉移。你已經解決了「系統會不會爆炸」的問題。

Chapter 4 前

狀態邏輯還不穩定

Chapter 8

現在可以升級互動

1

2

3

Chapter 6

核心功能完成驗收

如果在邏輯未穩之前就做互動升級，會發生三件事：UI 變成邏輯來源、Debug 時分不清錯在資料還是畫面、產品直接失控。



第一個關卡：狀態回退

目前 App #2 是單向的：todo → in-progress → done。

這非常安全，但你一定會遇到真實需求：做到一半發現要退回去怎麼辦？

產品視角的考量

- 回退會不會造成混亂？
- 使用者會不會誤觸？
- 是否需要確認機制？
- 回退會影響其他項目嗎？

Chapter 8 的做法

我們將示範「可以怎麼設計回退」，但不要求全班實作。

因為一旦加入回退，系統複雜度會明顯上升。這是一個產品決策，不是技術問題。

三原則仍然適用

即使進入互動升級，前面學的三原則一條都不能丟：

1 Backend First

回退不是 UI 行為，而是狀態轉移規則改變。Spec 必須先動。

2 Iterative

一次只加一種互動。不要同時做回退、拖拉和動畫。

3 Atomic Edit

避免 UI 重構災難。AI 最愛順便重構整個 render 邏輯。

拖拉功能為什麼最後才加？

拖拉（Drag & Drop）在視覺上很吸引人，但它其實只是：狀態轉移的另一種輸入方式。

它不是新功能，而是把「按鈕推進」換成「手勢推進」。

1 明確 Allowed Flow

先定義清楚允許的狀態轉移規則

2 統一轉移函式

所有互動呼叫同一個狀態轉移函式

3 拖拉作為觸發

拖拉只是其中一個觸發方式

❏ 如果先做拖拉，使用者會繞過驗證流程，不合法狀態更容易發生，UI 行為會反過來支配邏輯。

互動與狀態的正確架構

互動只是入口，狀態轉移才是產品核心。

1 User Interaction

使用者觸發動作（按鈕、拖拉等）

3 Data Update

更新資料模型

2 State Transition Function

統一的狀態轉移邏輯與驗證

4 UI Re-render

根據新狀態重新渲染畫面



你現在應該能回答這三個問題

這一章的角色是讓你看到產品「往上長」的可能性，而不是逼你現在就長到極限。你要帶走的不是拖拉程式碼，而是三個核心認知：

1 所有互動都是狀態轉移的不同入口

2 UI 永遠不該成為狀態真相

3 功能升級本質是 Spec 的進化

Chapter 9

資料持久化與多裝置

為什麼「資料存起來」會讓產品進入下一個世界？

當系統開始升級互動，使用者就會期待「東西不要消失」。

關掉瀏覽器資料不見，不是 bug，而是系統尚未進入產品下一階段的訊號。



資料持久化不只是技術決定

很多人會直覺這樣想：那我就把資料存起來就好了啊。但這一行看似簡單的決定，其實會同時改變三件事：

1. 資料的生命週期 — 從暫時到永久
2. 系統的責任邊界 — 從當下到未來
3. 產品對使用者的承諾 — 從試用到信任

這就是為什麼「存資料」一定要是一章，而不是一行 code。

系統責任的根本轉變

1 資料持久化之前

- 資料存在記憶體
- 重整頁面就消失
- 使用者心理預期低

系統只需要負責：現在這一次操作是對的。

2 資料持久化之後

- 下次打開還在
- 換裝置也還在
- 不會莫名其妙壞掉

這代表：系統開始要對「時間」負責。

層級一：localStorage（個人裝置）

優點

- 幾乎零門檻
- 不需帳號、不需後端
- 非常適合 MVP 驗證

代價

- 只能在同一台裝置
- 清除瀏覽器資料就沒了
- 無法跨裝置同步

📌 最適合：個人工具，快速驗證階段的產品

層級二：雲端資料庫（單人帳號）

1 優點

跨裝置存取，可以備份，為商業化鋪路，提供更好的使用者體驗

2 代價

需要帳號與權限系統，系統複雜度明顯上升，需要考慮資安與隱私

3 適合場景

個人產品，早期 SaaS，需要跨裝置同步的應用

層級三：多人協作資料層

這個層級的特性

- 優點：團隊協作，精細權限控管，企業級功能
- 代價：架構與成本明顯上升，需要複雜的權限邏輯

這個層級不在本課程範圍內，因為它涉及的是企業級架構設計，而非產品 MVP 階段需要處理的問題。

為什麼我們只示範 localStorage？

如果在這一章直接學 Firebase、SQL、API，你會以為：資料存起來 = 技術選型問題。

但事實是：資料存起來 = 產品承諾升級問題。

localStorage 的真正價值



專注核心



避免干擾



看清本質

資料持久化對 Spec 的影響

這一節非常重要，請你慢慢看。一旦資料會被保存，Spec 中必須多回答三件事：

1 什麼時候寫入？

明確定義觸發資料寫入的時機
點與條件

2 什麼時候讀取？

系統初始化、頁面載入、狀態
恢復的邏輯

3 哪些狀態下不應該覆寫？

避免資料衝突與意外覆蓋的保
護機制

❏ 如果這三件事沒寫清楚，AI 會幫你「好心補完」，然後系統就會亂。

localStorage 接入的正確順序

這一段以示範為主，不要求全班跟做。重點是理解概念與順序，而不是記住每一行程式碼。

1 資料初始化

先讀 localStorage，如果有資料就載入，沒有就用預設值。

2 狀態改變

使用者操作時，先更新記憶體中的資料。

3 同步寫回

記憶體資料改變後，立即或延遲寫回 localStorage。

4 UI 重新渲染

UI 根據記憶體中的資料重新渲染，確保一致性。

關鍵概念：localStorage 的定位

localStorage 不是資料來源，它只是資料的「備份層」

這個概念非常重要。記憶體中的資料才是真正的「當前狀態」，localStorage 只是用來「恢復」這個狀態的機制。如果搞混這個順序，就會產生各種同步問題。

✗ 每次 render 都覆寫 localStorage

這會造成不必要的效能負擔，也可能在讀取過程中產生競爭條件。

✗ UI 狀態跟資料狀態不一致

這是最常見的錯誤，會導致使用者看到的跟實際儲存的相同。

✗ 把 localStorage 當成即時資料來源

localStorage 是同步 API，過度依賴會造成效能問題。

這些錯誤，幾乎都是：沒有先想清楚「資料生命週期」。





你現在應該能回答這三個問題

1 為什麼資料持久化是產品層級的決定

它不只是技術實作，而是對使用者的承諾升級，會改變整個產品的責任邊界。

2 為什麼 localStorage 是安全的第一步

它讓你專注在資料流本身，而不被帳號、權限等複雜度分心，是最好的學習起點。

3 為什麼 Spec 必須跟著資料層一起升級

資料持久化後，Spec 必須明確定義寫入、讀取、衝突處理的規則，否則 AI 會亂補完。

當資料開始被保存，下一個一定會出現的問題是：

- 那使用者怎麼用？
- 我怎麼知道他在幹嘛？
- 他卡住了我該怎麼知道？

Chapter 10

真實上線後的產品調整

為什麼「能跑」不是終點，而是你第一次被現實打臉的開始

在 Chapter 9，你已經讓產品具備了一個重要能力：資料可以留下來。

這代表你的產品已經開始對「未來」負責了。

而一旦資料留下來，下一個一定會發生的事情是：使用者會開始用出你沒想過的方式。



上線後第一個會壞的通常不是技術

你以為會遇到的問題

- 程式錯誤
- 效能不夠
- 架構不穩

實際上第一個出現的

- 使用者看不懂
- 使用方式跟你想的不一樣
- 他卡住，但你不知道卡在哪

這不是 bug，是產品理解落差。

回饋的真正意義

真正有價值的回饋

不是使用者說了什麼

而是他做了什麼

行為痕跡：真實的產品訊號



停留時間異常

卡片長時間停在某個狀態，可能代表流程不清楚或操作門檻太高。



完成率低

很少有人把項目推到 Done，可能是「完成」的定義不符合使用者心理模型。



整理行為缺乏

使用者只新增不整理，可能代表整理的價值感不足，或操作成本太高。

這些都不是技術錯誤，是產品訊號。

為什麼不能直接改程式？

當你看到這些現象時，很容易下意識想：那我就幫他加個功能吧。

❏ 這是產品死亡最快的路。

因為你現在面對的不是功能缺失，而是：

- 流程是否清楚？
- 狀態是否符合使用者心理模型？
- Spec 是否反映真實使用情境？



上線後的正確調整順序

在這一章，你要養成一個產品級反射動作。正確順序是：

1 觀察行為

收集使用者的實際操作資料與行為模式

2 整理假設

從行為推測可能的困惑點或心理阻力

3 修改 Spec

先更新產品規格，重新定義規則與流程

4 再調整實作

基於新的 Spec 進程式修改

不是反過來。

為什麼一定要先動 Spec？因為一旦你跳過 Spec：

1. AI 會以舊世界觀幫你改 — 它不知道你看到了什麼新的使用情境
2. 你會得到「看似合理、實際混亂」的結果 — 表面上解決了問題，實際上埋下更多隱患。
3. 每改一次，Context 都會變得更不穩 — 系統的一致性逐漸瓦解。

Spec 的角色轉變

前面幾章的角色

護欄 — 限制 AI 的行為，確保修改在可控範圍內

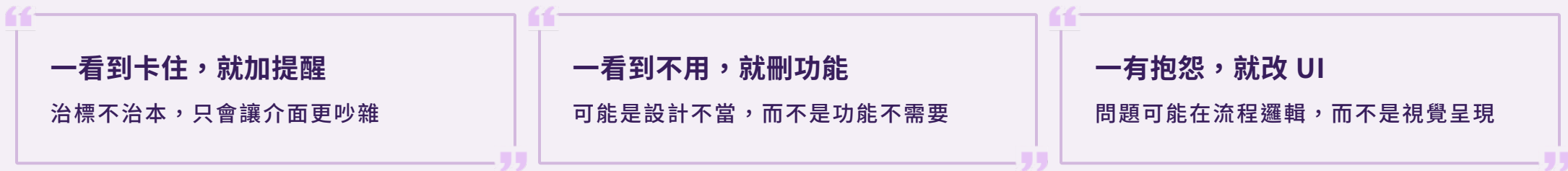
Spec 不只是「現在規則是什麼」，而是：

- 為什麼當初這樣設計
- 什麼情況下我們決定要改
- 改動的目標是什麼

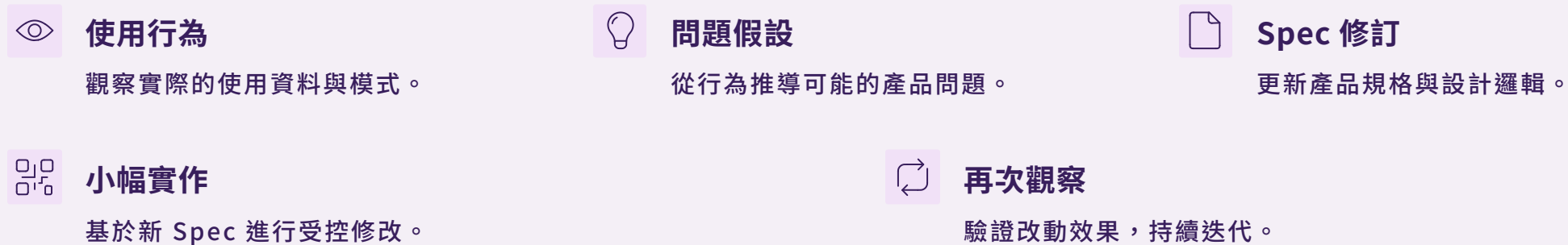
Chapter 10 的新角色

產品紀錄與決策日誌 — 記錄為什麼這樣設計，什麼情況下決定要改

常見的「錯誤修法」：它們在逃避真正的產品問題

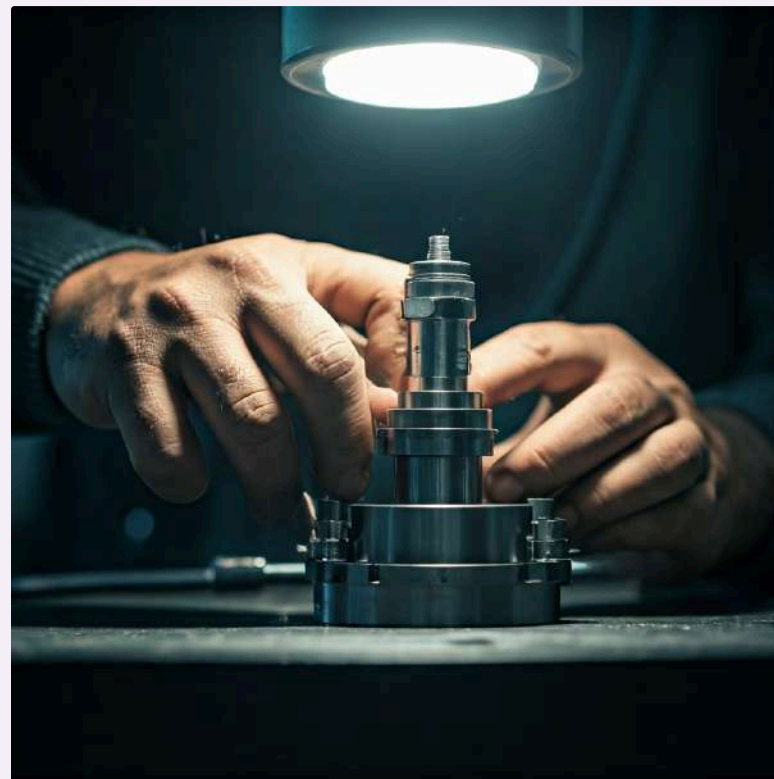


健康的產品調整循環



這個循環，才是：AI 能長期穩定協作的前提。

AI 課程的分水嶺



這一章教的是：克制、判斷、延遲滿足。這些能力，才是你之後能「用 AI 做大事」的關鍵。

你現在具備的能力

1 能做出產品的能力

2 能穩定修改的紀律

3 能從回饋中演化 Spec 的方法

接下來最後一個問題是：那 AI，未來到底該扮演什麼角色？是工具？是工程師？還是系統的一部分？

Chapter 11

AI × 產品的長期協作模式

從「我叫 AI 幫我寫」到「我們一起維持一個系統」



加速器 (Builder)

快速生成骨架、降低啟動門檻、把模糊想法變成可運作的東西



協作者 (Collaborator)

被 Spec 限制、修改有順序、在護欄內工作 — 這是產品開始穩定的階段



系統組件 (System Component)

參與理解、判斷、演化，被固定在流程中而非臨時救火



真正成熟的協作關鍵

不是 Prompt 寫得好

而是 AI 被放在對的位置

AI 在課程中的位置設計

1 Chapter 2：看 Context

2 Chapter 3：對齊 Spec

3 Chapter 4：受控修改

4 Chapter 8：風險預測

5 Chapter 10：回饋分析

你會發現一件事：AI 從來沒有被放在「決策者」的位置。這是關鍵。

把 AI 從救火隊拉回標準流程

✗ 錯誤的使用方式

- 卡住了 → 叫 AI
- 爆炸了 → 叫 AI
- 看不懂 → 叫 AI

這會讓 AI 永遠處在：高 Context、低理解、亂補完的狀態。

✓ 正確的進場時機

- 在改之前 → 幫你預測風險
- 在寫 Spec 時 → 幫你對齊理解
- 在上線後 → 幫你整理回饋

而不是在系統已經亂掉之後，期待它幫你「全部修好」。

Spec × AI：會一起老的組合

如果你只帶走一個長期做法，請帶走這個：

📄 Spec 不是寫給人看的，而是寫給「未來的你 × AI」看的

為什麼？因為半年後：

- 你會忘記當初為什麼這樣設計
- AI 也只看得到現在的檔案

Spec 是唯一能：保留設計理由、限制 Context 演化方向、防止產品變形的東西。

長期協作的最小閉環，是你未來所有 AI 協作的底盤



產品觀察



假設整理



Spec 修訂



受控修改 (AI)



再次觀察

如果哪一天你發現每次都直接跳到「改程式」、Spec 永遠追不上實作、AI 改完你不敢動，那代表你已經脫離這個循環了。

真正的 AI 素養是什麼？

很多人以為的 AI 素養

- 會不會下指令
- 會不會用新工具
- 會不會寫 Prompt

真正的 AI 素養

系統思維

- 判斷什麼時候該讓 AI 出手
- 判斷什麼時候該先停下來想
- 判斷哪些東西不能交給 AI



你現在站在哪裡？

如果你走到這裡，你已經具備：



把想法變成產品的能力



用 Spec 管住 Context 的能力



讓 AI 成為長期協作者的能力

這不是初學者的位置。